

Using `gam2formula`

Contents

1	Gaussian GAM	2
1.1	Simulate data and fit model with <code>mgcv</code>	2
1.2	Derive closed formulas for all supported smooth terms	3
1.3	Print derived formulas	3
1.3.1	Cubic regression spline for <code>x0</code>	3
1.3.2	B-spline for <code>x1</code>	3
1.3.3	P-spline for <code>x2</code>	4
1.4	Compare to internal predictions by <code>mgcv</code>	5
1.4.1	Cubic regression spline for <code>x0</code>	5
1.4.2	B-spline for <code>x1</code>	5
1.4.3	P-spline for <code>x2</code>	6
1.5	Print formulas in LaTeX	8
1.5.1	Cubic regression spline for <code>x0</code>	8
1.5.2	B-spline for <code>x1</code>	8
1.5.3	P-spline for <code>x2</code>	8
2	Bernoulli GAM	10
2.1	Fit model with <code>mgcv</code>	10
2.2	Print formulas in LaTeX	10
2.3	Making point predictions	11

Load packages:

```
library(mgcv)
library(gam2formula)
```

1 Gaussian GAM

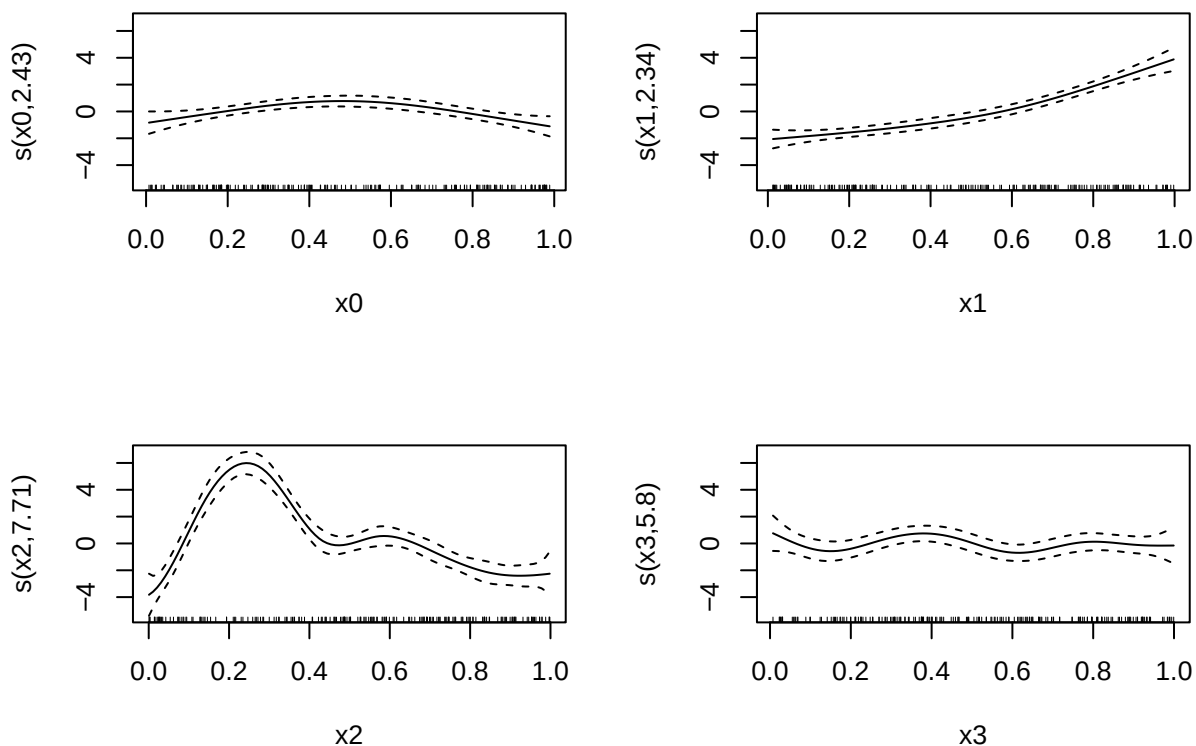
1.1 Simulate data and fit model with mgcv

Fit a GAM with mgcv to simulated data using B-splines, P-splines and cubic regression splines, among others:

```
set.seed(2)
dat <- gamSim(1, n = 200, dist = "normal", scale = 2, verbose = FALSE)
mod <- gam(y ~ s(x0, bs = "cr") +
            s(x1, bs = "bs", m = c(4, 2)) +
            s(x2, bs = "ps") +
            s(x3), data = dat)
```

Plot the smooth terms with mgcv's internal plot-function:

```
plot(mod, pages = 1)
```



1.2 Derive closed formulas for all supported smooth terms

Applying `gam2formula()` derives closed formulas for all supported smooth terms. Use `summary()` to get an overview:

```
mod_formulas <- gam2formula(mod)
summary(mod_formulas)

## gam2formula object with 3 derived formulas for: x0, x1, x2
## Original call:
## gam(formula = y ~ s(x0, bs = "cr") + s(x1, bs = "bs", m = c(4, 2)) + s(x2, bs = "ps") +
##       s(x3), data = dat)
##
## Extracted smooth terms:
## term type
## x0    cr
## x1    bs
## x2    ps
```

1.3 Print derived formulas

To print a derived formula, use `print()` with the desired `term`.

1.3.1 Cubic regression spline for x0

Let's look at the formula for the predictor `x0`, modeled with a cubic regression spline:

```
print(mod_formulas, term = "x0")

## # A tibble: 12 x 4
##   term range bfun      coef
##   <chr> <chr> <chr>    <dbl>
## 1 x0    1      1      1.42
## 2 x0    1      (x0-0.00710903806611896)/0.981793117476627 -0.154
## 3 x0    1      abs((x0-0.00710903806611896)/0.981793117476627)^3 -1.18
## 4 x0    1      abs((x0-0.115639718844452)/0.981793117476627)^3  2.24
## 5 x0    1      abs((x0-0.184966982135342)/0.981793117476627)^3 -11.1
## 6 x0    1      abs((x0-0.289176538431396)/0.981793117476627)^3  5.73
## 7 x0    1      abs((x0-0.388033331578804)/0.981793117476627)^3  1.63
## 8 x0    1      abs((x0-0.509289973027383)/0.981793117476627)^3  6.76
## 9 x0    1      abs((x0-0.666686570970342)/0.981793117476627)^3  4.23
## 10 x0   1      abs((x0-0.810351680860751)/0.981793117476627)^3 -7.74
## 11 x0   1      abs((x0-0.903263577078986)/0.981793117476627)^3 -0.673
## 12 x0   1      abs((x0-0.988902155542746)/0.981793117476627)^3  0.136
```

1.3.2 B-spline for x1

Let's look at the formula for the predictor `x1`, modeled with a B-spline of degree 4:

```
print(mod_formulas, term = "x1")

## # A tibble: 14 x 4
##   term range      bfun      coef
##   <chr> <chr>    <chr>    <dbl>
## 1 x1    x1 < 0.0116218442101963 1      -2.06
## 2 x1    x1 < 0.0116218442101963 (x1-0.01162~ 2.50
## 3 x1    x1 >= 0.0116218442101963 & x1 <= 0.998757580574136 1      -2.06
## 4 x1    x1 >= 0.0116218442101963 & x1 <= 0.998757580574136 (x1-0.01162~ 2.50
## 5 x1    x1 >= 0.0116218442101963 & x1 <= 0.998757580574136 ((x1-0.0116~ -0.268
## 6 x1    x1 >= 0.0116218442101963 & x1 <= 0.998757580574136 ((x1-0.0116~ 2.28
## 7 x1    x1 >= 0.0116218442101963 & x1 <= 0.998757580574136 ((x1-0.0116~ 5.17
```

```
## 8 x1      x1 >= 0.0116218442101963 & x1 <= 0.998757580574136 pmax((x1-0.~ -13.6
## 9 x1      x1 >= 0.0116218442101963 & x1 <= 0.998757580574136 pmax((x1-0.~ 33.0
## 10 x1     x1 >= 0.0116218442101963 & x1 <= 0.998757580574136 pmax((x1-0.~ -69.2
## 11 x1     x1 >= 0.0116218442101963 & x1 <= 0.998757580574136 pmax((x1-0.~ 51.1
## 12 x1     x1 >= 0.0116218442101963 & x1 <= 0.998757580574136 pmax((x1-0.~ 4.67
## 13 x1     x1 > 0.998757580574136                                1                -6.27
## 14 x1     x1 > 0.998757580574136                                (x1-0.01162~ 10.2
```

1.3.3 P-spline for x2

Let's look at the formula for the predictor x2, modeled with a P-spline of degree 3:

```
print(mod_formulas, term = "x2")
```

```
## # A tibble: 14 x 4
##   term range                                bfun      coef
##   <chr> <chr>                                <chr>      <dbl>
## 1 x2    x2 < -0.000334179506637167            1      -3.84e0
## 2 x2    x2 < -0.000334179506637167          (x2+0.00~ 1.71e1
## 3 x2    x2 >= -0.000334179506637167 & x2 <= 0.999061476917006 1      -3.84e0
## 4 x2    x2 >= -0.000334179506637167 & x2 <= 0.999061476917006 (x2+0.00~ 1.71e1
## 5 x2    x2 >= -0.000334179506637167 & x2 <= 0.999061476917006 ((x2+0.0~ 4.62e2
## 6 x2    x2 >= -0.000334179506637167 & x2 <= 0.999061476917006 ((x2+0.0~ -1.61e3
## 7 x2    x2 >= -0.000334179506637167 & x2 <= 0.999061476917006 pmax((x2~ 1.45e3
## 8 x2    x2 >= -0.000334179506637167 & x2 <= 0.999061476917006 pmax((x2~ 1.55e3
## 9 x2    x2 >= -0.000334179506637167 & x2 <= 0.999061476917006 pmax((x2~ -2.42e3
## 10 x2   x2 >= -0.000334179506637167 & x2 <= 0.999061476917006 pmax((x2~ 1.35e3
## 11 x2   x2 >= -0.000334179506637167 & x2 <= 0.999061476917006 pmax((x2~ -2.32e2
## 12 x2   x2 >= -0.000334179506637167 & x2 <= 0.999061476917006 pmax((x2~ -1.78e2
## 13 x2   x2 > 0.999061476917006                1      -5.92e0
## 14 x2   x2 > 0.999061476917006          (x2+0.00~ 3.67e0
```

1.4 Compare to internal predictions by mgcv

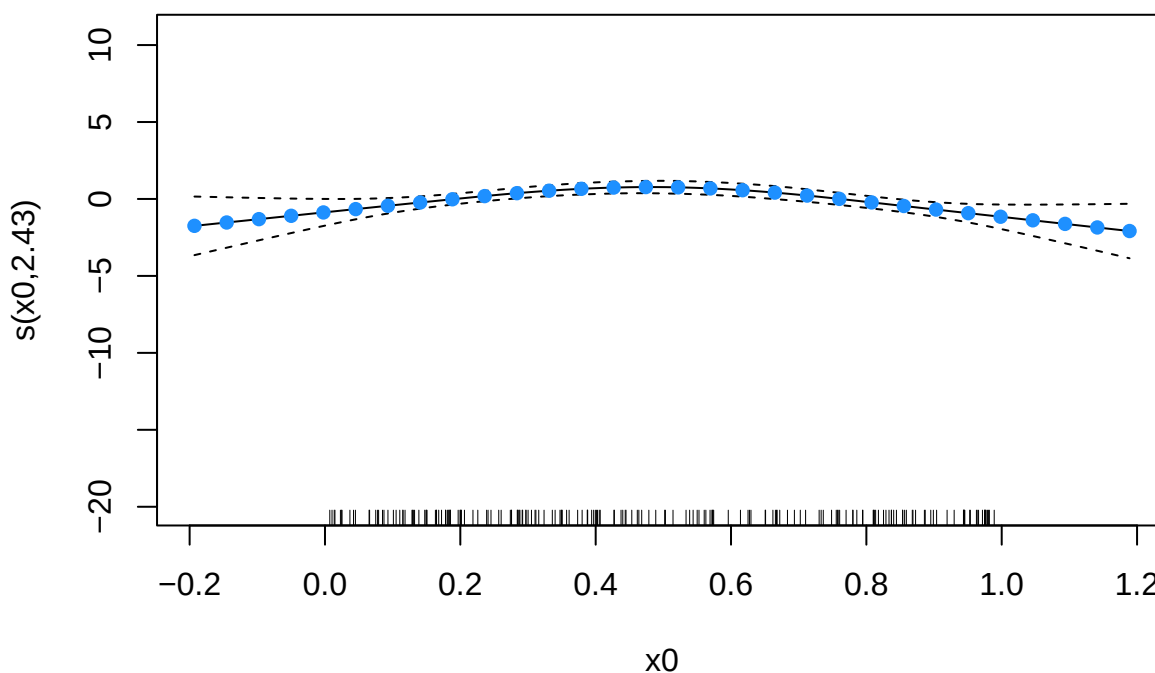
As a quick visual check, we can make predictions for a vector of new predictor values based on our obtained formulas, using `predict()`, and compare it to the internal predictions plot from `mgcv`. We deliberately choose new values of the predictor partially outside of the observed range to also show the expected behavior for extrapolation (which is linear by `mgcv`'s default).

1.4.1 Cubic regression spline for x_0

```
# generate new predictor data
x0_newmin <- min(dat$x0) - 0.2
x0_newmax <- max(dat$x0) + 0.2
x0_new <- seq(x0_newmin, x0_newmax, length.out = 30)

# compute the linear predictor from coefficient table for new data points
y <- predict(mod_formulas, term = "x0", newdata = data.frame(x0 = x0_new))

# plot comparison
plot(mod, select = 1, xlim = c(x0_newmin, x0_newmax))
points(x0_new, y, col = "dodgerblue", pch = 16)
```



1.4.2 B-spline for x_1

```
# generate new predictor data
x1_newmin <- min(dat$x1) - 0.2
x1_newmax <- max(dat$x1) + 0.2
x1_new <- seq(x1_newmin, x1_newmax, length.out = 30)

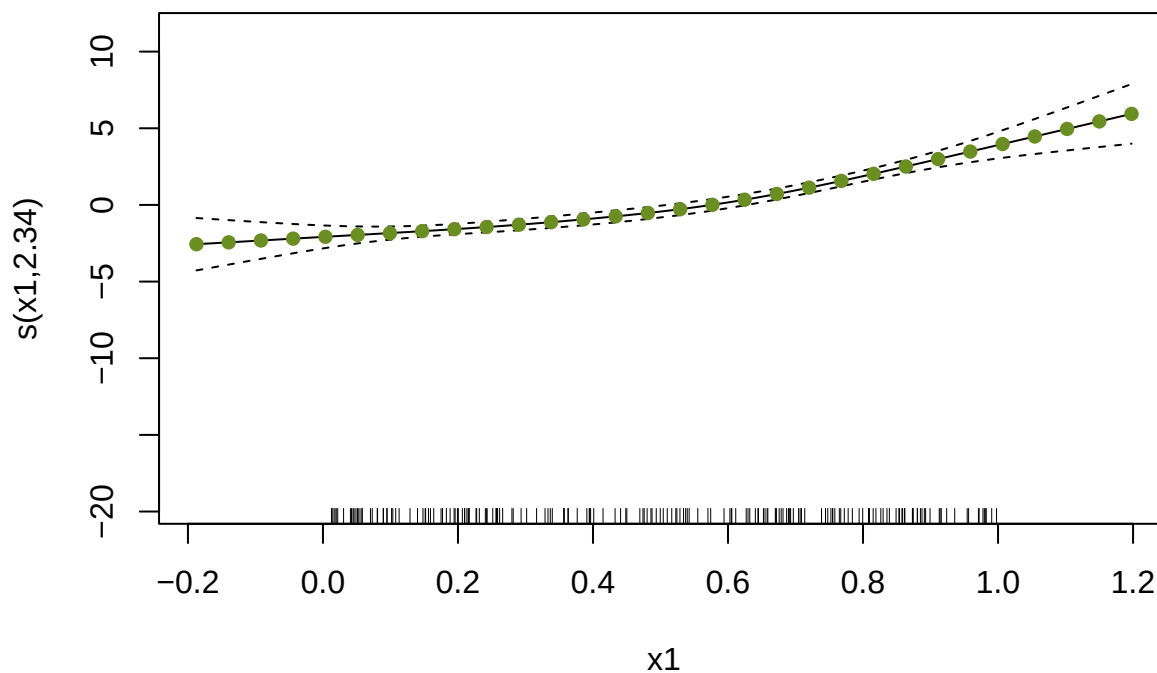
# compute the linear predictor from coefficient table for new data points
```

```

y <- predict(mod_formulas, term = "x1", newdata = data.frame(x1 = x1_new))

# plot comparison
plot(mod, select = 2, xlim = c(x1_newmin, x1_newmax))
points(x1_new, y, col = "olivedrab", pch = 16)

```



1.4.3 P-spline for x2

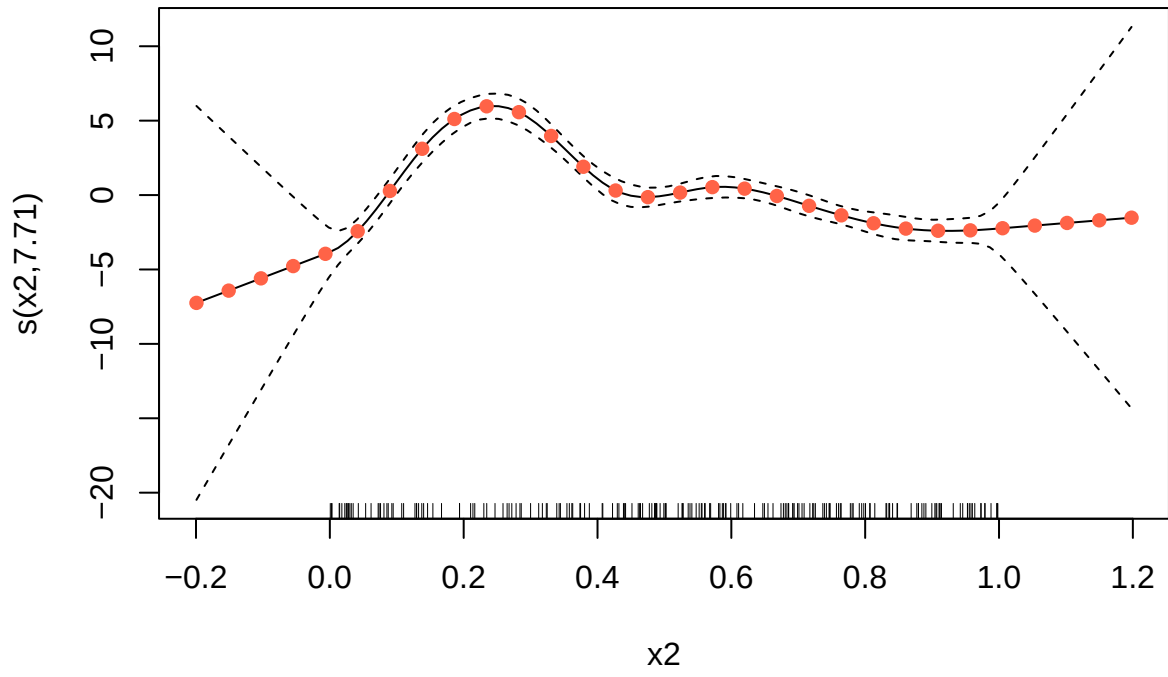
```

# generate new predictor data
x2_newmin <- min(dat$x2) - 0.2
x2_newmax <- max(dat$x2) + 0.2
x2_new <- seq(x2_newmin, x2_newmax, length.out = 30)

# compute the linear predictor from coefficient table for new data points
y <- predict(mod_formulas, term = "x2", newdata = data.frame(x2 = x2_new))

# plot comparison
plot(mod, select = 3, xlim = c(x2_newmin, x2_newmax))
points(x2_new, y, col = "tomato", pch = 16)

```



1.5 Print formulas in LaTeX

We can also print the formulas in LaTeX using the `print()` function. The resulting LaTeX code can be pasted directly into a LaTeX editor of your choice. When using `results = 'asis'` for the chunk options, the LaTeX code will be compiled directly in an RMarkdown script like the current vignette.

1.5.1 Cubic regression spline for x_0

```
print(mod_formulas, term = "x0", format = "latex")
```

Basis function	Coefficient
1	1.42403652364965
$(x_0 - 0.00710903806611896)/0.981793117476627$	-0.153965035713098
$ (x_0 - 0.00710903806611896)/0.981793117476627 ^3$	-1.17881318146407
$ (x_0 - 0.115639718844452)/0.981793117476627 ^3$	2.2449977122272
$ (x_0 - 0.184966982135342)/0.981793117476627 ^3$	-11.1462285047155
$ (x_0 - 0.289176538431396)/0.981793117476627 ^3$	5.73232835496988
$ (x_0 - 0.388033331578804)/0.981793117476627 ^3$	1.63217146394009
$ (x_0 - 0.509289973027383)/0.981793117476627 ^3$	6.76165413618237
$ (x_0 - 0.666686570970342)/0.981793117476627 ^3$	4.22636113394682
$ (x_0 - 0.810351680860751)/0.981793117476627 ^3$	-7.73551280189963
$ (x_0 - 0.903263577078986)/0.981793117476627 ^3$	-0.673130516029408
$ (x_0 - 0.988902155542746)/0.981793117476627 ^3$	0.136172202844637

1.5.2 B-spline for x_1

```
print(mod_formulas, term = "x1", format = "latex")
```

Basis function	Coefficient
For $x_1 < 0.0116218442101963$	
1	-2.05812362705562
$(x_1 - 0.0116218442101963)/0.98713573636394$	2.50479812297575
For $x_1 \geq 0.0116218442101963$ and $x_1 \leq 0.998757580574136$	
1	-2.05812362705562
$(x_1 - 0.0116218442101963)/0.98713573636394$	2.50479812297575
$((x_1 - 0.0116218442101963)/0.98713573636394)^2$	-0.268315727065016
$((x_1 - 0.0116218442101963)/0.98713573636394)^3$	2.28244558964653
$((x_1 - 0.0116218442101963)/0.98713573636394)^4$	5.16635697402061
$\max\{(x_1 - 0.17614446693752)/0.98713573636394, 0\}^4$	-13.6278548297668
$\max\{(x_1 - 0.340667089664843)/0.98713573636394, 0\}^4$	33.0214607128028
$\max\{(x_1 - 0.505189712392166)/0.98713573636394, 0\}^4$	-69.1561824672
$\max\{(x_1 - 0.66971233511949)/0.98713573636394, 0\}^4$	51.145209600941
$\max\{(x_1 - 0.834234957846813)/0.98713573636394, 0\}^4$	4.66968167994937
For $x_1 > 0.998757580574136$	
1	-6.2663615146121
$(x_1 - 0.0116218442101963)/0.98713573636394$	10.1569695518605

1.5.3 P-spline for x_2

```
print(mod_formulas, term = "x2", format = "latex")
```

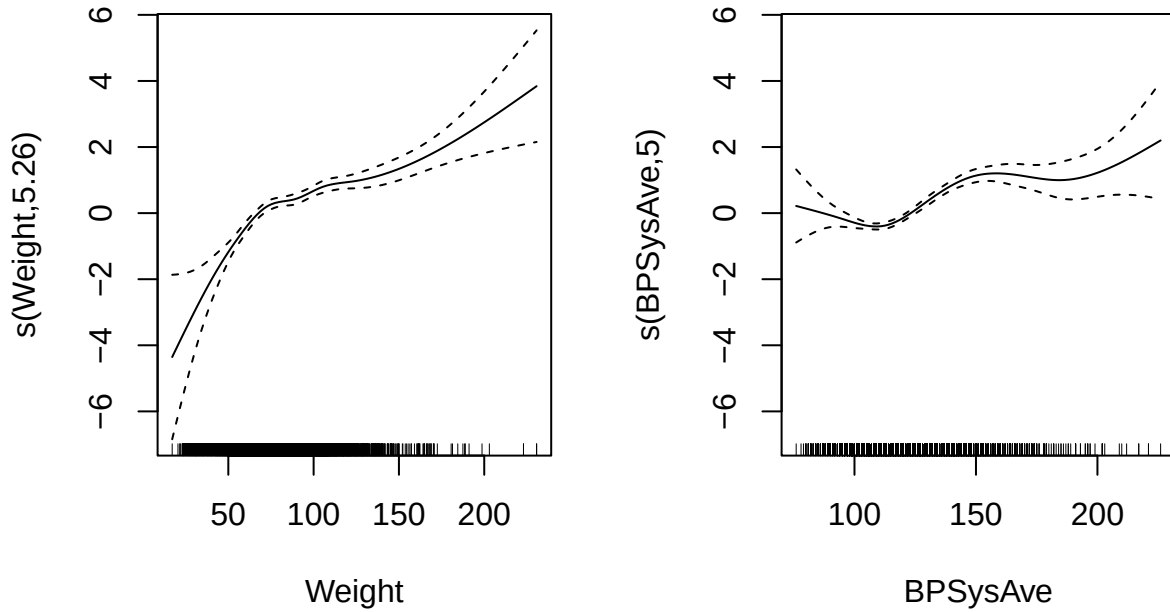
Basis function	Coefficient
For $x_2 < -0.000334179506637167$	
1	-3.84167734368608
$(x_2 + 0.000334179506637167)/0.999395656423643$	17.0996992360188
For $x_2 \geq -0.000334179506637167$ and $x_2 \leq 0.999061476917006$	
1	-3.84167734368608
$(x_2 + 0.000334179506637167)/0.999395656423643$	17.0996992360188
$((x_2 + 0.000334179506637167)/0.999395656423643)^2$	461.74003154946
$((x_2 + 0.000334179506637167)/0.999395656423643)^3$	-1606.4098847757
$\max\{(x_2 - 0.142436628553883)/0.999395656423643, 0\}^3$	1454.52296373677
$\max\{(x_2 - 0.285207436614404)/0.999395656423643, 0\}^3$	1549.26623382312
$\max\{(x_2 - 0.427978244674924)/0.999395656423643, 0\}^3$	-2423.06817853228
$\max\{(x_2 - 0.570749052735445)/0.999395656423643, 0\}^3$	1354.80994458219
$\max\{(x_2 - 0.713519860795965)/0.999395656423643, 0\}^3$	-232.252336863033
$\max\{(x_2 - 0.856290668856486)/0.999395656423643, 0\}^3$	-178.496145747438
For $x_2 > 0.999061476917006$	
1	-5.91779908549238
$(x_2 + 0.000334179506637167)/0.999395656423643$	3.66698708696888

2 Bernoulli GAM

2.1 Fit model with mgcv

As a real-world example, we use data from the US National Health and Nutrition Examination Study as provided in the NHANES package. We predict binary diabetes status (`Diabetes`) using cubic regression splines for weight (`Weight`) and systolic blood pressure (`BPSysAve`) and plot the estimated effects.

```
library(NHANES)
mod <- gam(Diabetes ~ s(Weight, bs = "cr", k = 8) + s(BPSysAve, bs = "cr", k = 8),
           family = "binomial", data = NHANES)
plot(mod, pages = 1)
```



2.2 Print formulas in LaTeX

```
mod_formulas <- gam2formula(mod)
print(mod_formulas, term = "Weight", format = "latex")
```

Basis function	Coefficient
1	-4.1482448064011
$(\text{Weight} - 17.1)/213.6$	15.2950081383837
$ (\text{Weight} - 17.1)/213.6 ^3$	-47.2691961112198
$ (\text{Weight} - 41.8428571428571)/213.6 ^3$	69.6000645489843
$ (\text{Weight} - 57.7857142857143)/213.6 ^3$	-192.261095751992
$ (\text{Weight} - 73.4285714285714)/213.6 ^3$	524.93885106577
$ (\text{Weight} - 89.0714285714286)/213.6 ^3$	-589.296527717194
$ (\text{Weight} - 104.714285714286)/213.6 ^3$	348.312785391889
$ (\text{Weight} - 122.457142857143)/213.6 ^3$	-118.087900233124
$ (\text{Weight} - 230.7)/213.6 ^3$	4.06301880690493

```
print(mod_formulas, term = "BPSysAve", format = "latex")
```

Basis function	Coefficient
1	-2.03449596389381
$(\text{BPSysAve} - 76)/150$	1.79646084806713
$ (\text{BPSysAve} - 76)/150 ^3$	-6.3523634931521
$ (\text{BPSysAve} - 94.8571428571429)/150 ^3$	89.553804224143
$ (\text{BPSysAve} - 112.714285714286)/150 ^3$	-171.861774171888
$ (\text{BPSysAve} - 130.571428571429)/150 ^3$	58.0735538097126
$ (\text{BPSysAve} - 148.428571428571)/150 ^3$	60.6084513685048
$ (\text{BPSysAve} - 166.285714285714)/150 ^3$	12.3649638839796
$ (\text{BPSysAve} - 185.142857142857)/150 ^3$	-56.38497074523
$ (\text{BPSysAve} - 226)/150 ^3$	13.9983351239278

2.3 Making point predictions

To get the predicted probability of diabetes for two newly observed individuals using `predict()`, we need to build the linear predictor from its three components (an intercept term, the weight term, and the systolic blood pressure term) and apply the inverse link function:

```
newdata <- data.frame(Weight = c(70, 130), BPSysAve = c(105, 115)) # new observations

linpred <- coef(mod)["(Intercept)"] + # global intercept
  predict(mod_formulas, "Weight", newdata) + # linear predictor term for weight
  predict(mod_formulas, "BPSysAve", newdata) # linear predictor term for systolic blood pressure

plogis(linpred)
```

```
## [1] 0.04770912 0.11703142
```

The result is equivalent to the predicted probabilities from `mgcv::predict.gam()`:

```
as.numeric(mgcv::predict.gam(mod, newdata, type = "response"))
```

```
## [1] 0.04770912 0.11703142
```